Aula 13 — Aprofundamento em Analisadores LR

Compiladores I

FACOM - Faculdade de Computação

Bloco 4: Análise Sintática

Duração estimada: 70–80 minutos

Revisão da Aula 12

- Deslocamento-redução: pilha + shift/reduce;
- Conflitos shift-reduce e reduce-reduce.

Pergunta rápida

O que a família de parsers LR promete resolver de forma sistemática, em comparação com um shift-reduce "ingênuo"?

Objetivos de aprendizagem

- Construir o conjunto de itens LR(0) e o autômato de itens para uma gramática simples;
- Construir uma tabela de análise SLR (ACTION/GOTO) a partir do autômato de itens;
- Comparar o poder de reconhecimento das análises LL(1) e LR.

Sumário da aula

Itens LR(0)

Tabela SLR

O que é um item LR(0)

Definição

Um item LR(0) é uma produção com um marcador (•) indicando até onde a análise já "reconheceu" o lado direito.

Exemplo —

Para `S -> A B`, os itens possíveis são: `S -> • A B`, `S -> A • B`, `S -> A B •` (item completo, indica possibilidade de redução).

Construindo o autômato de itens

- Cada **estado** do autômato é um conjunto de itens (fechamento — *closure*);
- **Closure**: se um item tem o marcador antes de um não-terminal A, adicione os itens iniciais de todas as produções de A;
- **Transições (goto):** avançar o marcador sobre um símbolo leva a um novo estado (conjunto de itens).

Pergunta rápida

Por que a operação de *closure* é necessária ao construir cada estado do autômato de itens?

Exemplo de construção

Exemplo — Gramática aumentada: `S' -> S`, `S -> '(' S ')' | ID`

Estado inicial (fechamento de `S' -> • S`):

- `S' -> • S`
- `S -> • '(' S ')'`
- `S -> • ID`

Ao ver o símbolo `ID`, transiciona para um novo estado contendo o item `S -> ID •` (item completo $\Rightarrow$ candidato a redução).

Sumário da aula

Itens LR(0)

Tabela SLR

Tabela de análise SLR: ACTION e GOTO

Duas tabelas

- **ACTION[estado, terminal]:** shift para outro estado, reduce por uma produção, ou aceitar;
- **GOTO[estado, não-terminal]:** para qual estado ir após uma redução.

Regra SLR para reduzir: um item completo `A -> `α $\bullet$ gera uma ação `reduce` para cada terminal em $FOLLOW(A)$ — por isso reaproveitamos os conjuntos FOLLOW da Aula 10!

Resolvendo conflitos com a tabela

- Se, para o mesmo estado e terminal, existe shift e reduce simultaneamente $\Rightarrow$ conflito shift-reduce na construção SLR;
- Se existem dois itens completos com o mesmo terminal em FOLLOW no mesmo estado $\Rightarrow$ conflito reduce-reduce;
- Gramáticas que geram tabelas sem conflitos são chamadas **SLR(1)**.

LL(1) x LR: comparando o poder de reconhecimento

	LL(1)	**LR (SLR/LALR/LR(1))**
Direção da construção	topo $\rightarrow$ base	base $\rightarrow$ topo
Recursão à esquerda	proibida	permitida
Classe de gramáticas	mais restrita	mais ampla
Implementação manual	fácil (recursiva)	difícil (usa tabelas)

Toda gramática LL(1) também pode ser reconhecida por um parser LR, mas o contrário não é verdade.

Desafio em sala

Desafio em sala

Para a gramática aumentada `S' -> S`, `S -> 'a' S | 'b'`: construa o estado inicial (fechamento de `S' -> •  S`) e determine para quais símbolos existem transições a partir desse estado.

Síntese da aula

- Itens LR(0) registram o "progresso" do reconhecimento de uma produção;
- O autômato de itens (via closure e goto) origina as tabelas ACTION/GOTO;
- A regra SLR usa FOLLOW para decidir reduções; LR aceita uma classe maior de gramáticas que LL(1).

Próxima aula: projeto de um analisador sintático usando ferramentas (Yacc/Bison).

Teste rápido — Aula 13

Questões teóricas

1. O que representa o marcador • em um item LR(0)?
2. Como a regra SLR decide para quais terminais uma redução deve ser aplicada?

Questão prática

3. Liste todos os itens LR(0) da produção `S -> '(' S ')'`.